```
>>> "\xF1".encode('ASCII')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
UnicodeEncodeError: 'ascii' codec can't encode character '\xf1' in position 0: ordinal not in range(128)
>>> "\xF1".encode('utf-8')
b'\xc3\xb1'
>>> "\xF1".encode('latin-1')
b'\xf1'
>>>
```

Figure 2: Difference in UTF-8 and Latin-1 encoding

encoding methods in Python. We use the *encode()* function with the name of the encoding method as the argument. As you can see, there is no difference in the output for the string A, as it is a common character that can be represented by the same byte in all the methods. However, this may not be the case for other characters, such as symbols, emojis, or foreign languages.

To understand encoding method differences, let's examine how they represent characters using bytes. A byte is a unit of data that consists of 8 bits, each of which can have a value of 0 or 1. Different encoding methods use different combinations of bits to map characters to bytes, depending on the range and the standard of the method. ASCII is one of the oldest and simplest encoding methods, which uses 7 bits to represent 128 characters, such as letters, numbers, punctuation marks, and control codes. ASCII is a signed method, which means that the first bit is reserved for indicating the sign of the byte, either positive or negative. Therefore, ASCII can only use the remaining 7 bits to encode characters, which limits its range to 2^7 = 128 characters. ASCII can't encode characters beyond this range, such as symbols, emojis, or foreign languages.

UTF-8, an advanced universal encoding method, uses variable-length bytes to represent over a million characters from different languages and scripts. It is an unsigned method, which means that it does not use the first bit for the sign of the byte, but for indicating the length of the byte. Therefore, UTF-8 can use all 8 bits to encode characters, which increases its range to 2^8 = 256 characters for a single byte. However, UTF-8 can also use more than one byte to encode characters that require more bits, such as two bytes for 2^16 = 65,536 characters, three bytes for 2^24 = 16,777,216 characters, or four bytes for 2^32 = 4,294,967,296 characters.

Latin-1, also known as ISO-8859-1, is another encoding method. It uses 8 bits to represent 256 characters, mostly from Western European languages, such as French, German, or Spanish. Latin-1 is also an unsigned method, which means that it can use all 8 bits to encode characters, which gives it the same range as UTF-8 for a single byte. However, Latin-1 does not use variable-length bytes, which means that it cannot encode characters that require more than 8 bits, such as Chinese, Arabic, or Hindi. Latin-1 is also not compatible with UTF-8. This may cause errors or corruption when converting between the two methods.

In Figure 2, we can see some characters differing in their representations in UTF-8 and Latin-1, with the latter having a much greater scope. Hence, not all encodings give the same bytes.

While Latin-1 consists of most of the characters, another encoding form called Base64 allows you to encode images, files and other media into bytes. Base64 decoding works by reversing the encoding process. It converts the encoded text string into groups of 4 characters, each of which corresponds to one of the 64 values in the Base64 alphabet. These values are then converted back to 6 bits, and the resulting bits are concatenated to form the original binary data.

In Python, Base64 is used to convert encoded string into Base64 bytes. This means that characters unable to be represented by encoding methods like ASCII, Latin-1 and UTF-8 get shown as '??' but get a valid meaning once encoded with Base64. In this way, as shown in Figure 3, emojis, images, files and string are converted into the binary for the computer to understand. END

```
>>> import base64
>>> "😊".encode()
b'??'
>>> "😊".encode('ASCII')
b'??'
>>> "😊".encode('UTF-8')
b'??'
>>> "😊".encode('latin-1')
b'??'
>>> base64.b64encode("😊".encode())
b'Pz8='
>>>
```

Figure 3: Explanation of Base64 encoding of non-ASCII characters

By: Anisha Ghosh

The author is a cybersecurity researcher and an active contributor to open source communities and repositories. She is interested in developing novel, scalable and secure systems.